

# **Digital System Design**

## **Term Project Presentation**

### **Team 6**

---

2026.06.11

# Contents



1

**Design Goal & Baseline**

2

**Architecture Differentiation**

3

**RTL Optimization Strategies**

4

**Results & Conclusion**

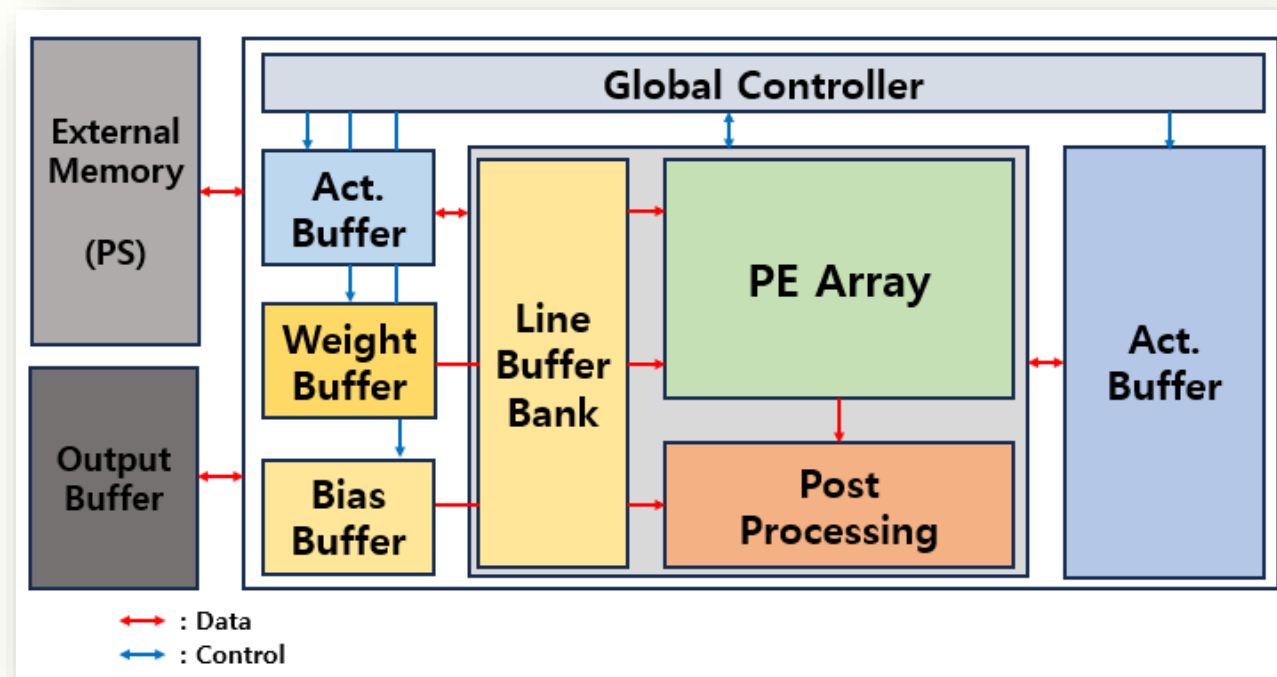
# 1 Design Goal & Baseline

※ AI Inference

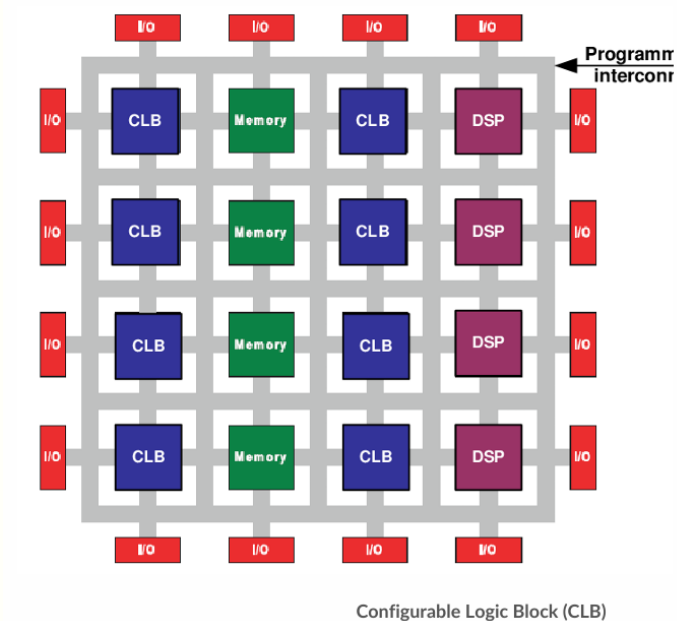
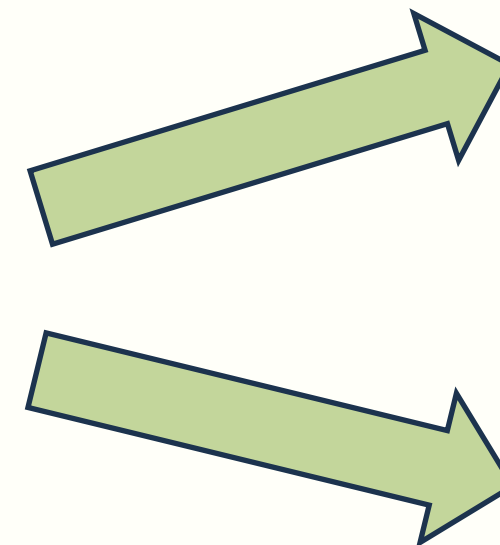
“Latency” Is the Bottleneck



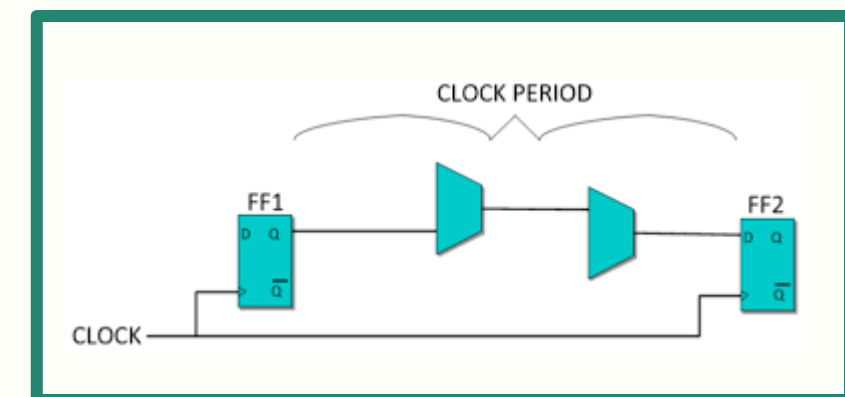
## ► Baseline: Recursive Architecture



Primary focus: lower latency

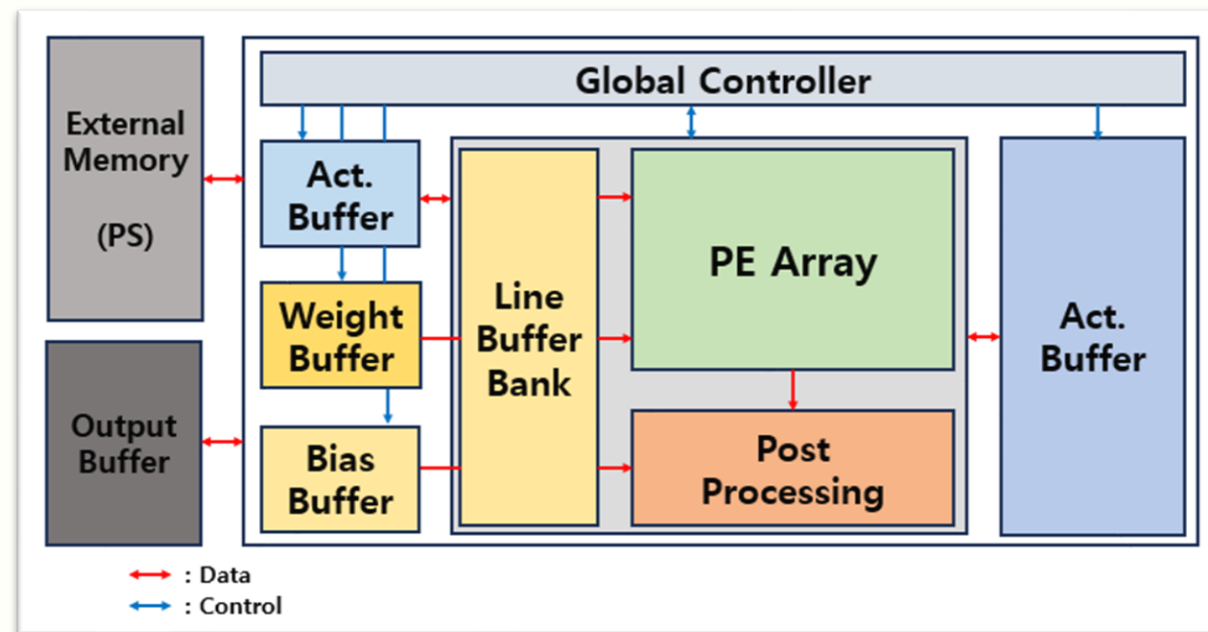


Resource

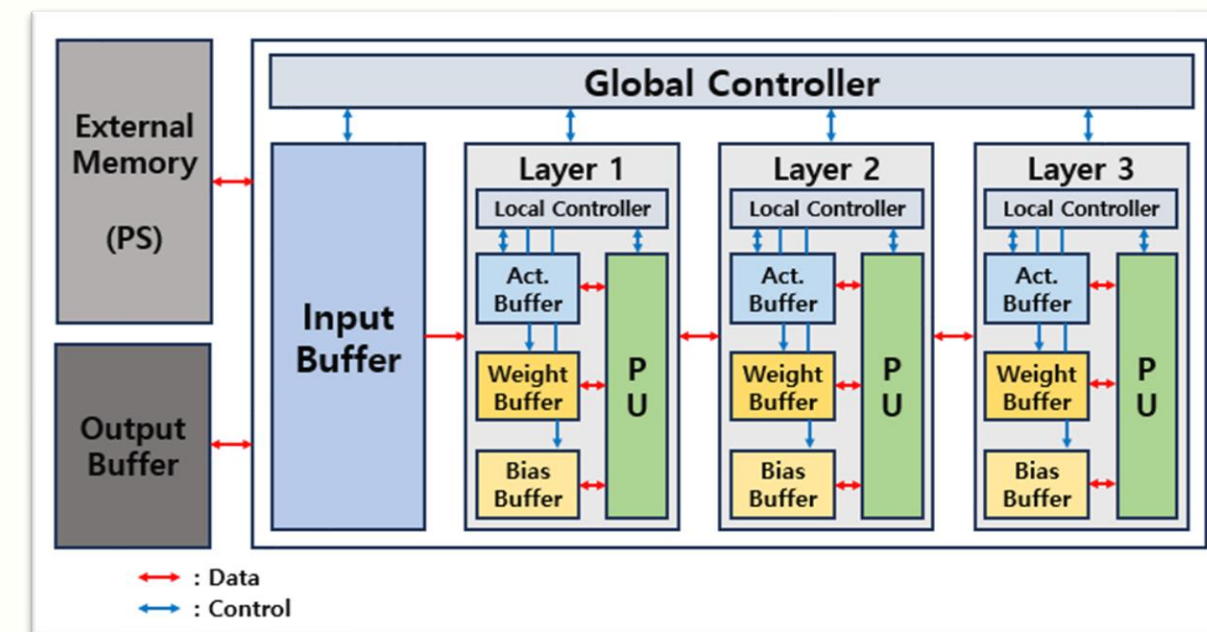


Latency

## 2 Architecture Differentiation



Our Recursive Structure (4\_2, 8\_8)



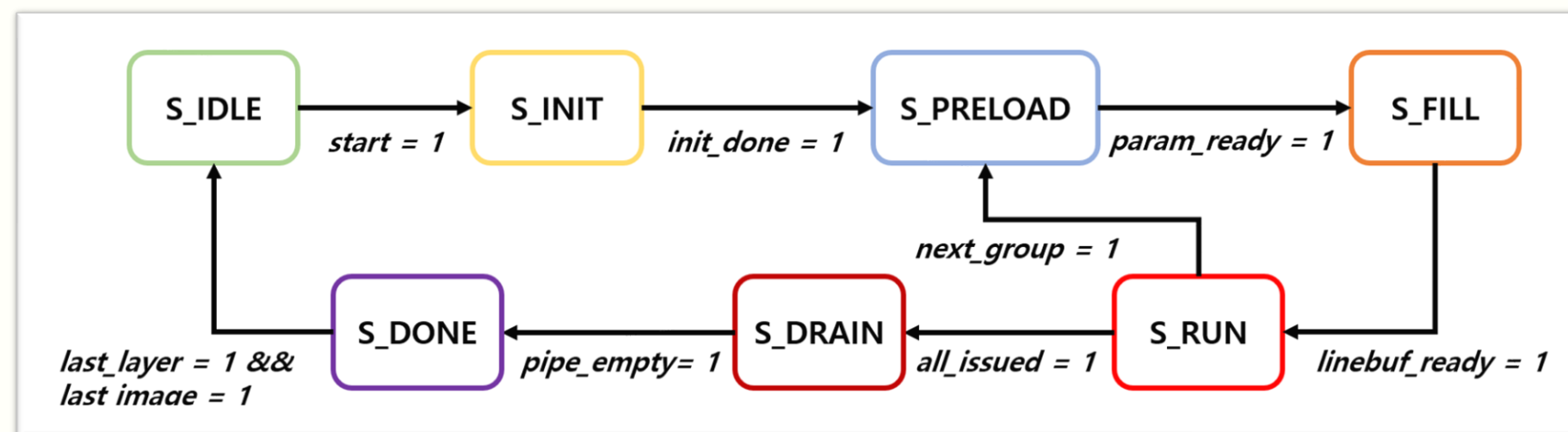
Our Streamline Structure (8\_4)

TABLE 1 Design Configurations

| DESIGN          | CHANNEL CONFIGURATION | ARCHITECTURE TYPE | MAIN DESIGN PURPOSE                                                  |
|-----------------|-----------------------|-------------------|----------------------------------------------------------------------|
| <b>SRCNN_88</b> | 8 / 8                 | Recursive         | Channel-wise parallel baseline and resource-cost reference           |
| <b>SRCNN_42</b> | 4 / 2                 | Recursive         | 2-pixel horizontal latency reduction using available resource margin |
| <b>SRCNN_84</b> | 8 / 4                 | Streamline        | Layer-overlapped processing for lower total latency                  |

## 2 Architecture Differentiation

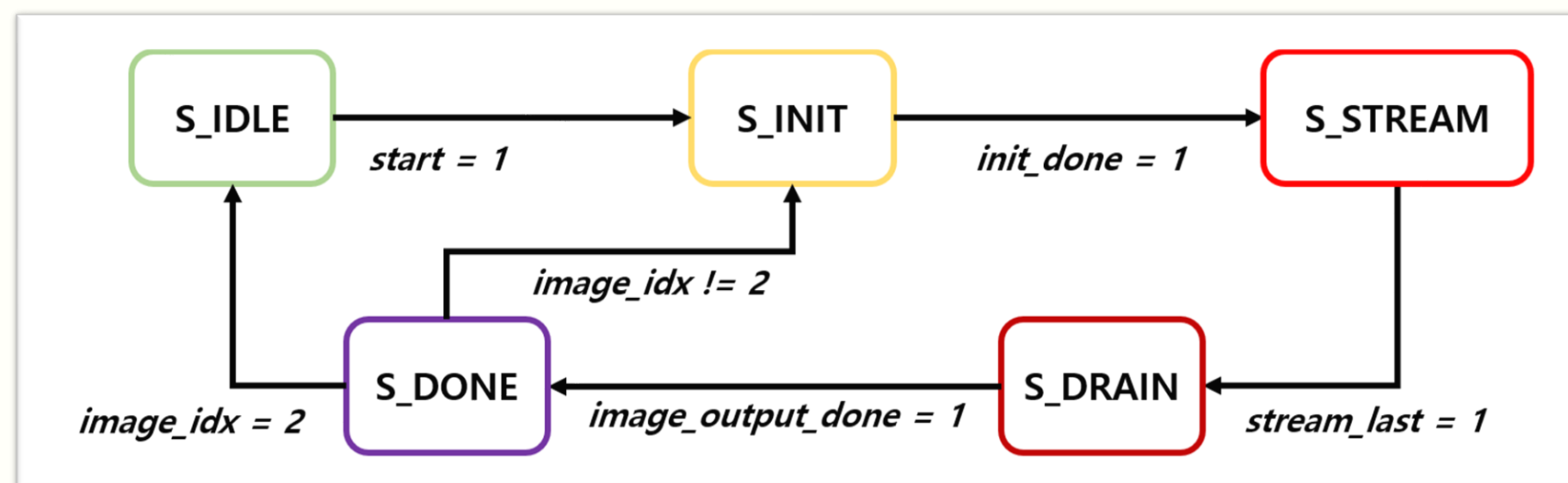
### ✂ Finite State Machine (FSM)



#### ◆ Key States

- PRELOAD: Computation preparation state
- FILL: Window filling state
- RUN: Active computation state
- DRAIN: Pipeline draining state

Layer-by-layer execution



#### ◆ Key States

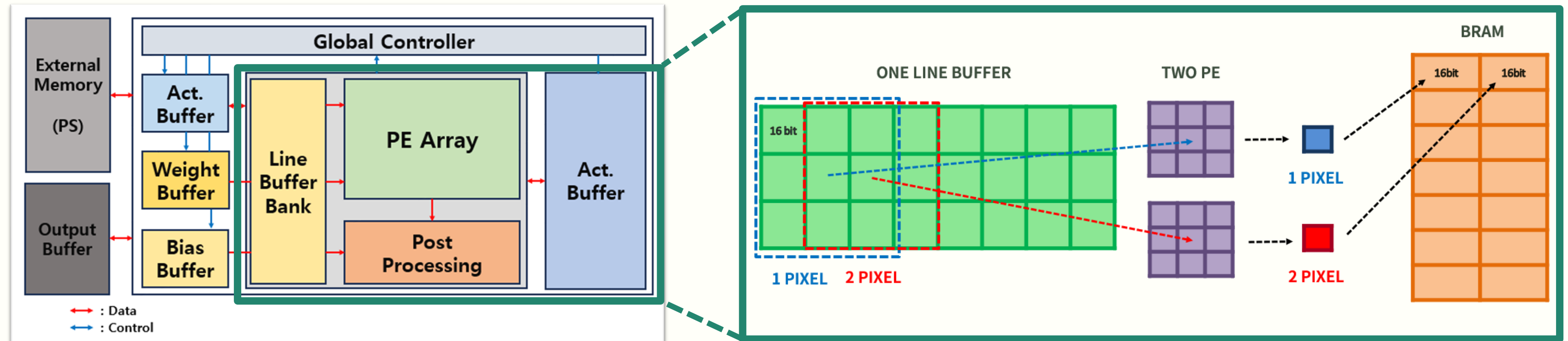
- STREAM: Streaming computation state
- DRAIN: Pipeline draining state

Stage overlap



## 2 Architecture Differentiation

### 1. SRCNN 4\_2 Design



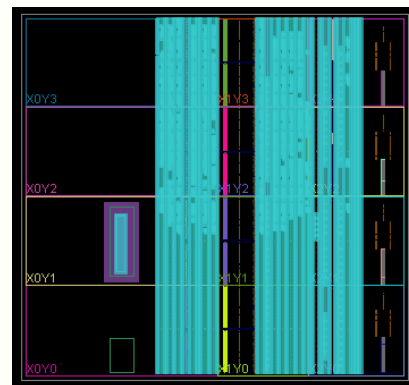
Initial design: Weight Stationary  
Channel-wise Parallelism

Horizontal PE duplication → 2 pixels  
per cycle (lower latency)

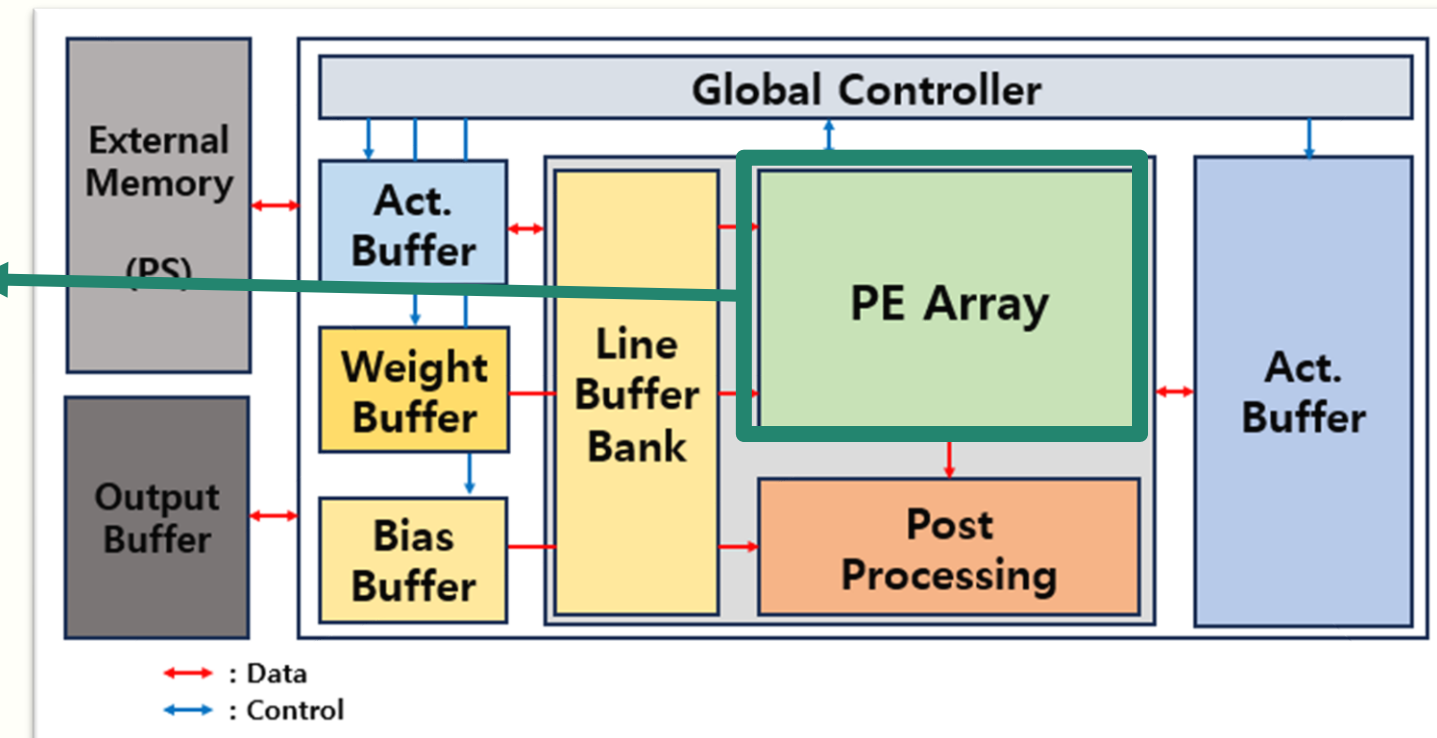
# 2 Architecture Differentiation

## 2. SRCNN 8\_8 Design

Layer 2 requires 64 PEs for channel-wise parallelism



Synthesis succeeded with 64 PEs



### DECISION SRCNN\_88: Rationale for Selecting 64PE over 32PE

| Architecture              | LAYER2 PE | LAYER2 Runs | Latency (Cycles) | LUT     | DSP |
|---------------------------|-----------|-------------|------------------|---------|-----|
| 32PE Split Computation    | 32        | 2           | 552,000          | 99,400  | 289 |
| 64PE Parallel Computation | 64        | 1           | 413,188          | 104,330 | 577 |

#### Trade-off Summary

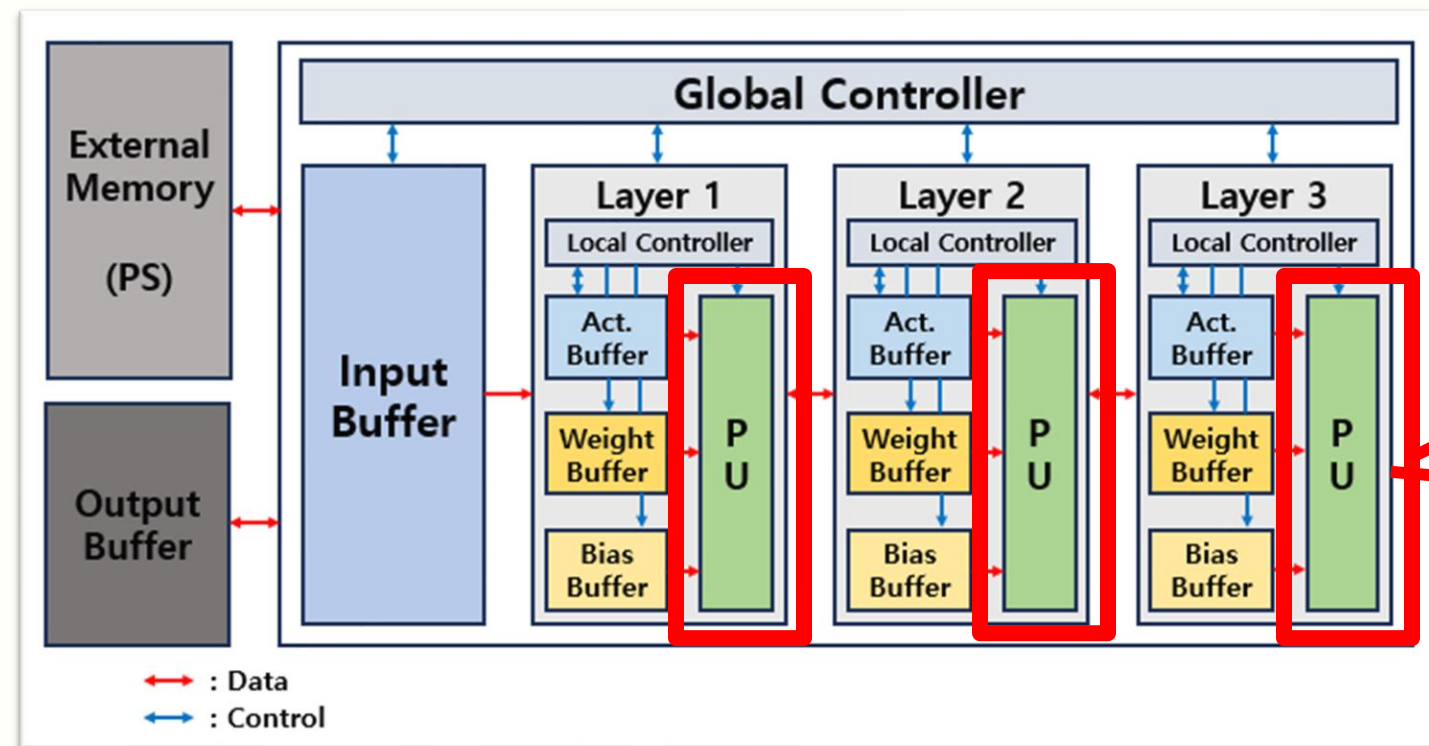
|                     |             |        |
|---------------------|-------------|--------|
| Latency Improvement | <div></div> | -25.1% |
| LUT Increase        | <div></div> | +5.0%  |
| DSP Utilization     | <div></div> | 46.2%  |
| Layer2 Runs         | <div></div> | 2 -> 1 |

64 PEs selected → minimized latency

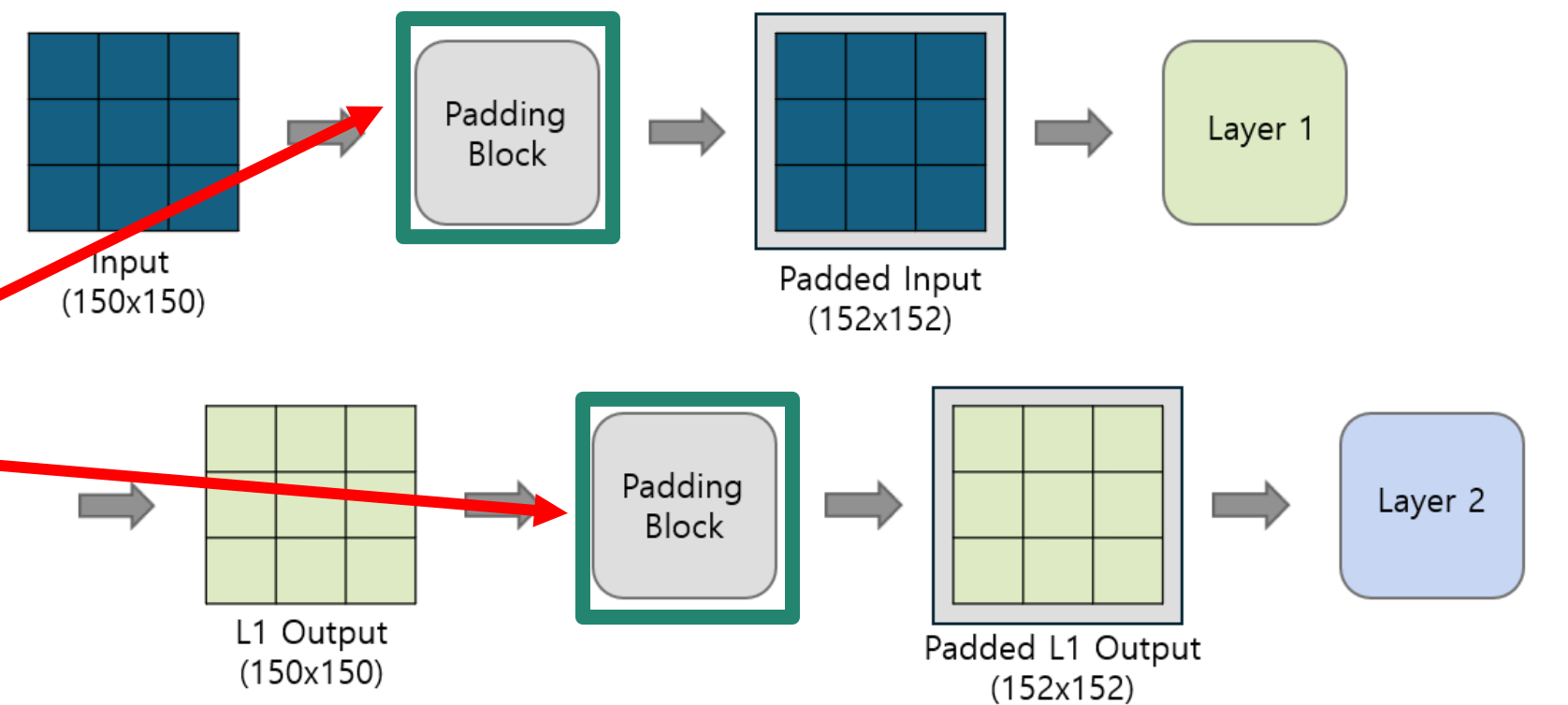


## 2 Architecture Differentiation

### 3. SRCNN 8\_4 Design



Layer-to-layer operation  
overlap required



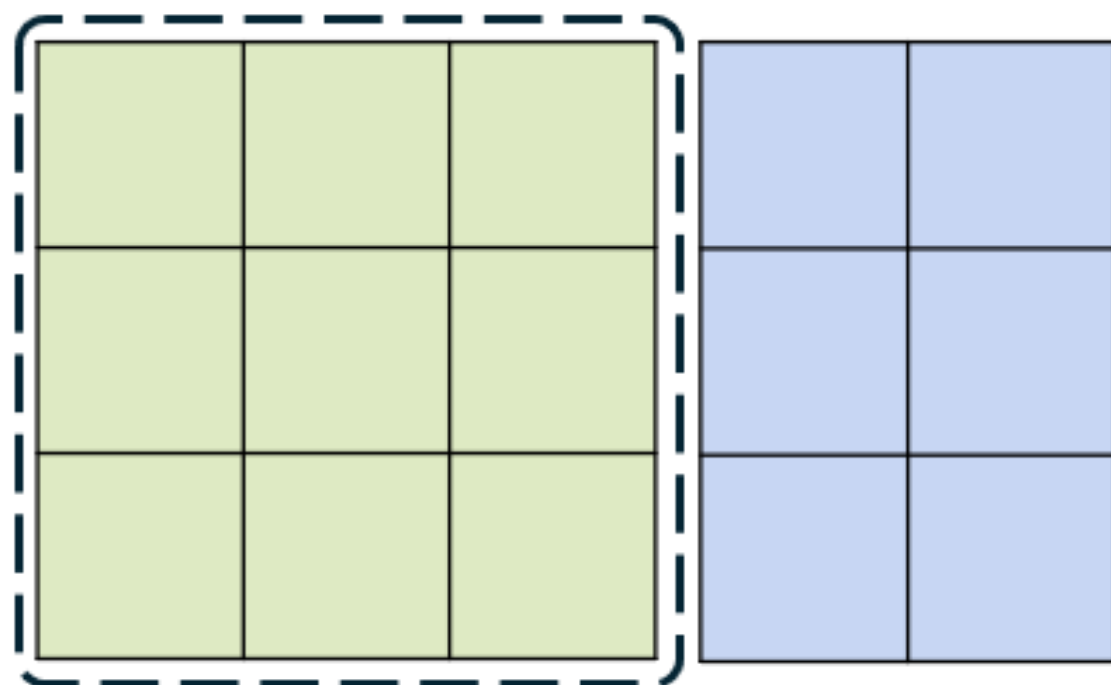
Padding modules in each Processing  
Unit → valid feature maps



### 3 RTL Optimization Strategies

#### ※ Scan-Window Line Buffer

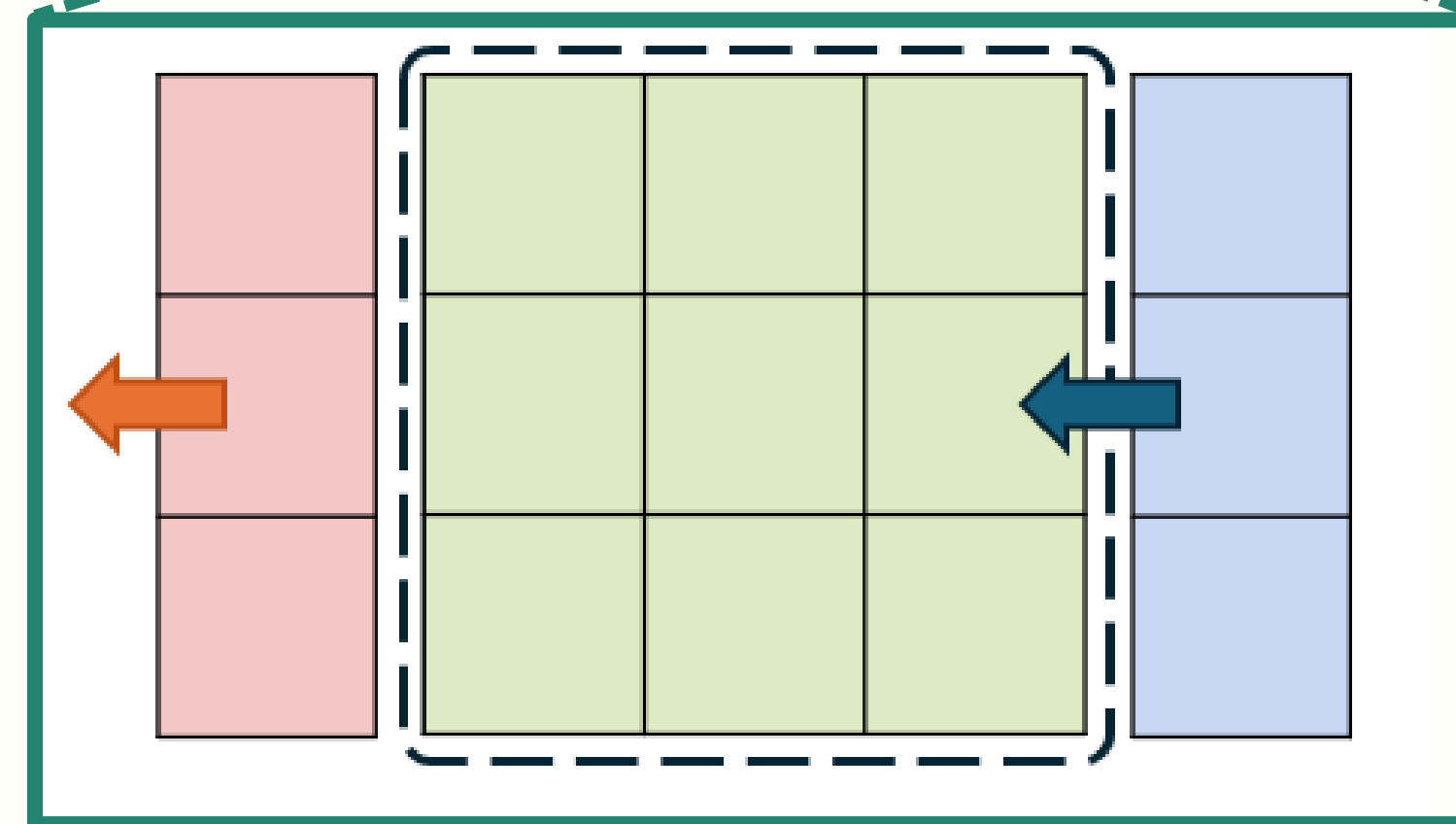
LUT usage reduced by approx. 15%  
(Resource ↓)



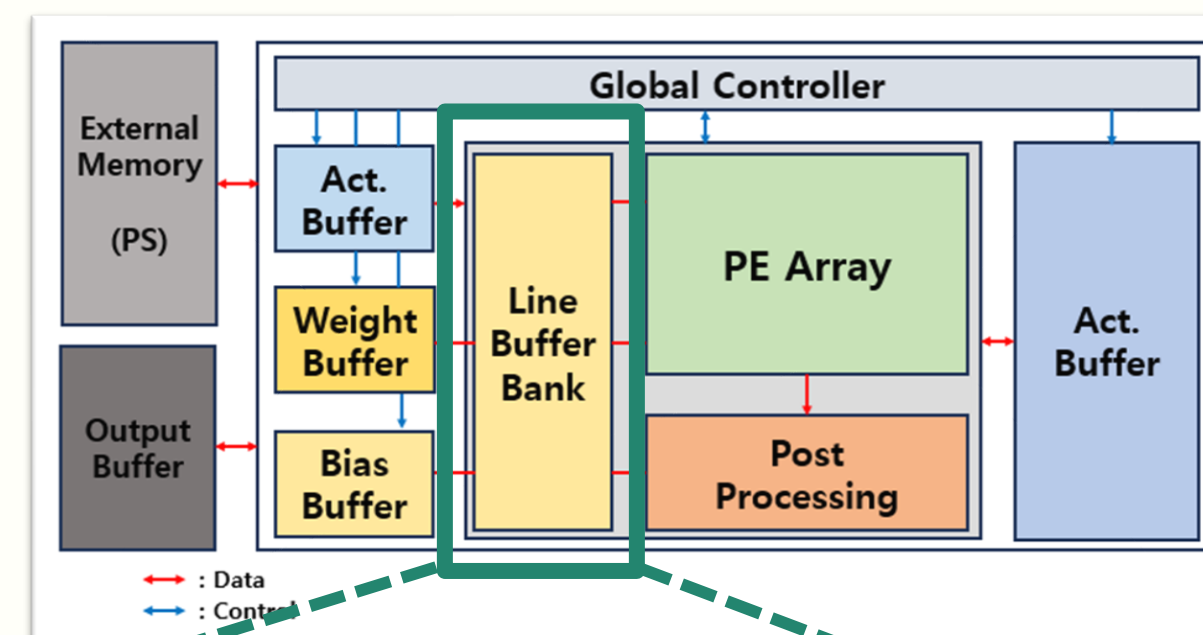
3x3 Window

Initial Line Buffer

Improvement  
Direction

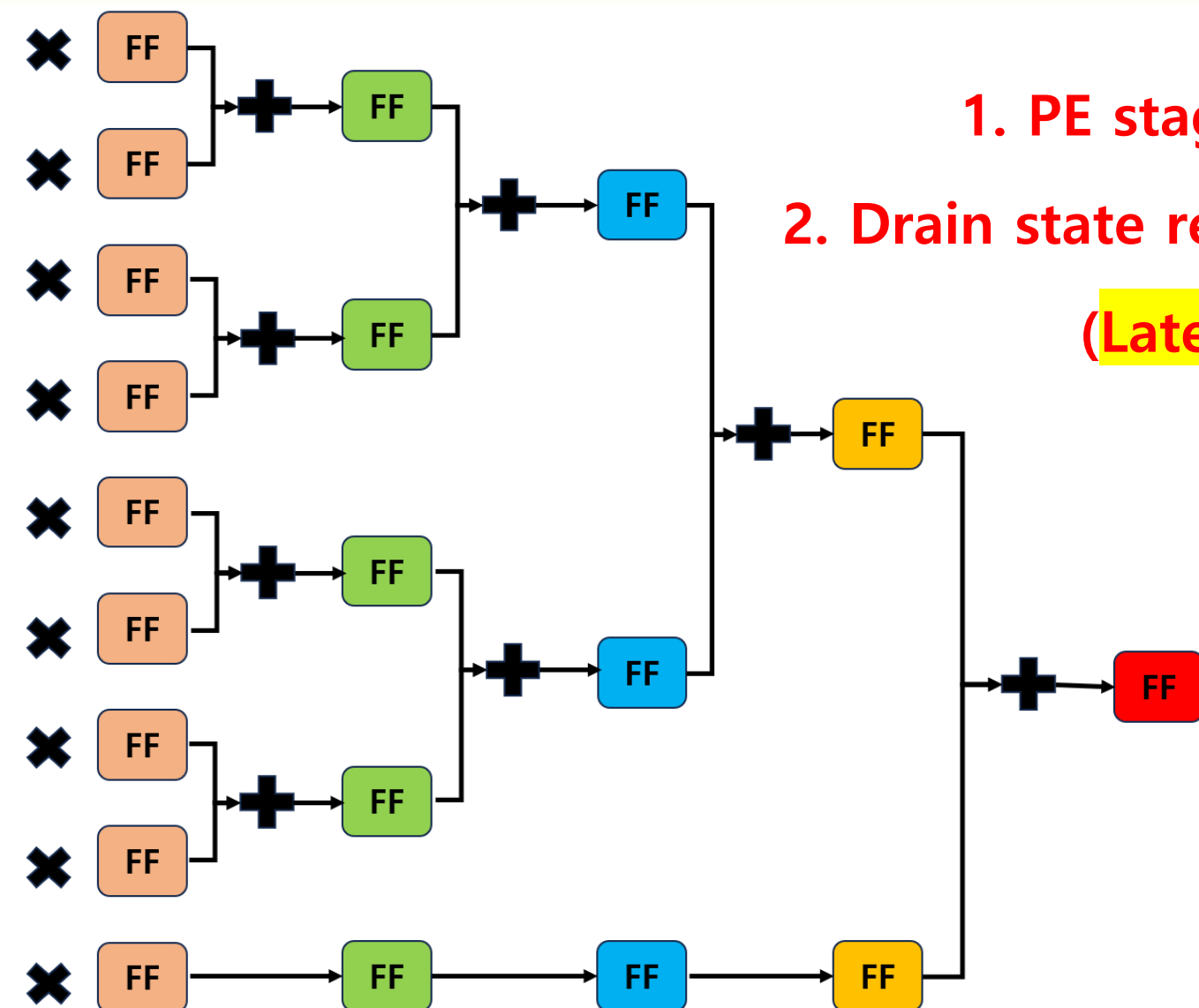


Improved Window Generation



### 3 RTL Optimization Strategies

#### ※ PE Pipeline Optimization

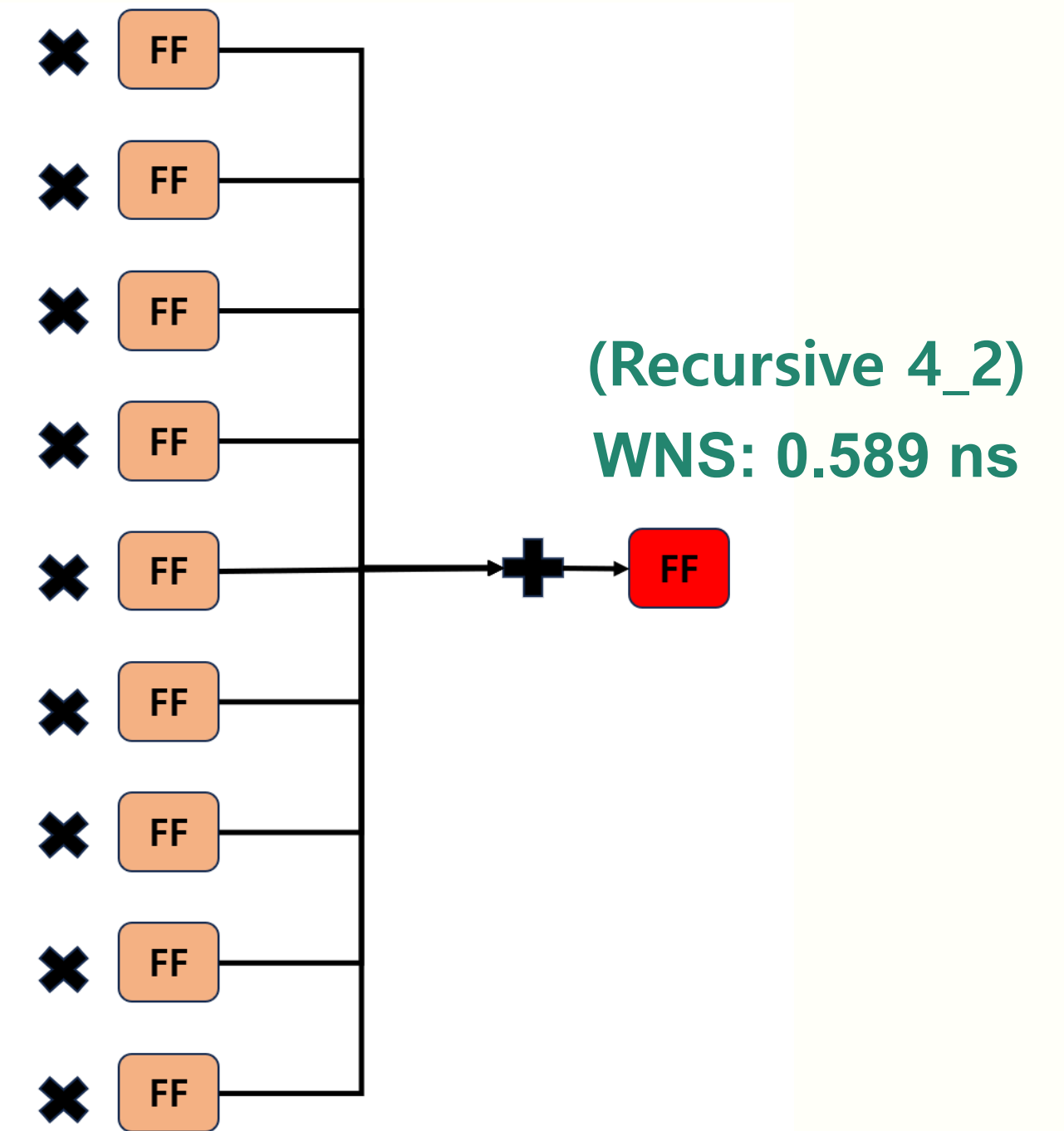


5-stage Pipeline

- 1. PE stages reduced
- 2. Drain state reduced by 3 cycles  
(Latency ↓)



Improvement  
Direction



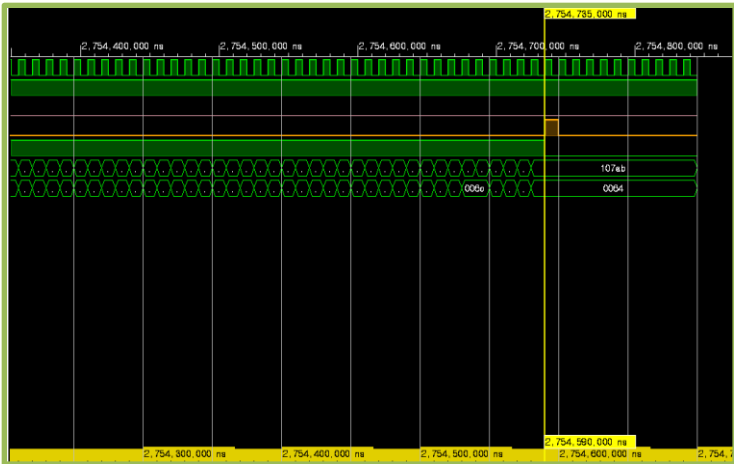
2-stage Pipeline

(Recursive 4\_2)  
WNS: 0.589 ns

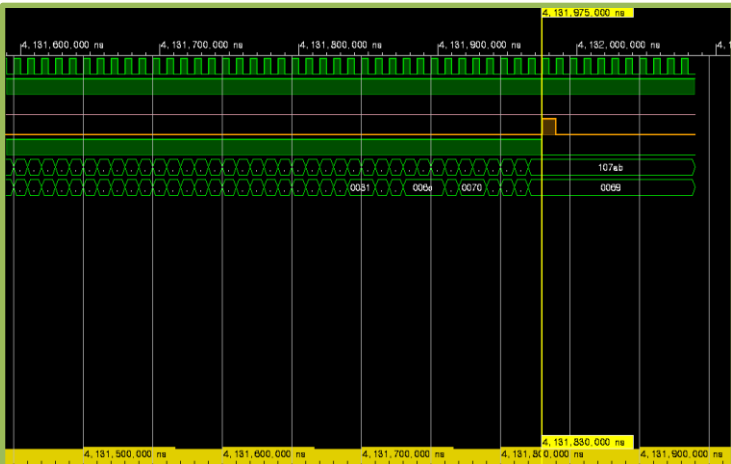
# 4 Results & Conclusion

## ✂ Latency Comparison

SRCNN 4\_2



SRCNN 8\_8



SRCNN 8\_4

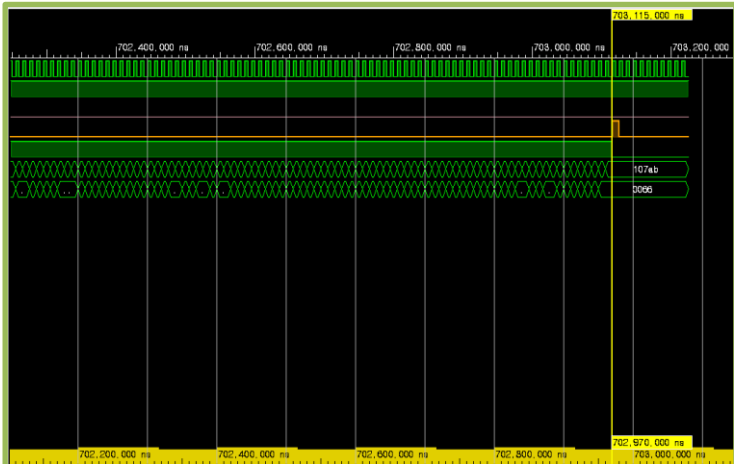


TABLE 4 Functional Verification

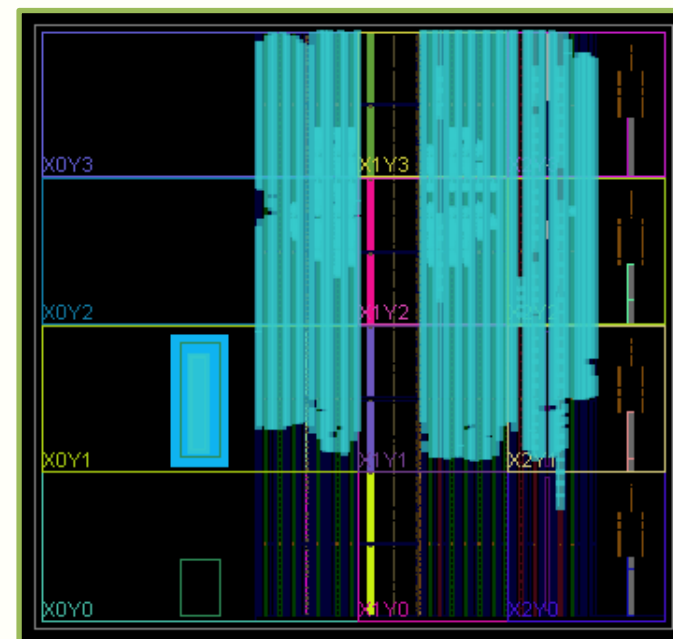
| DESIGN   | ARCHITECTURE                     | TOTAL LATENCY (CYCLES) | TOTAL LATENCY (US) | WRITES | MISMATCHES |
|----------|----------------------------------|------------------------|--------------------|--------|------------|
| SRCNN_42 | Recursive, 2-pixel, channel-wise | 275,459                | 2754.59            | 67,500 | 0          |
| SRCNN_88 | Recursive, channel-wise          | 413,183                | 4131.83            | 67,500 | 0          |
| SRCNN_84 | Streamline                       | 70,297                 | 702.97             | 67,500 | 0          |

Latency is measured from the rising i\_start request to the final o\_done after all three 150 x 150 images.

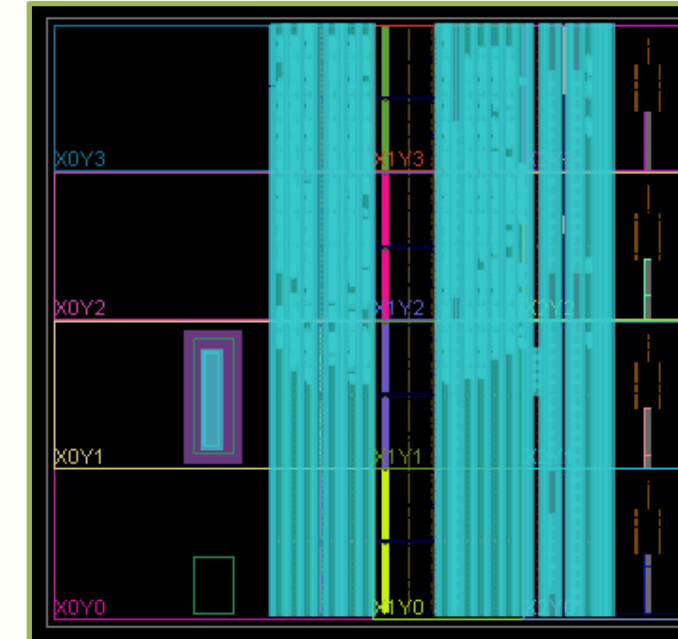
# 4 Results & Conclusion

## ✂ Resource Utilization

SRCNN 4\_2



SRCNN 8\_8



SRCNN 8 4

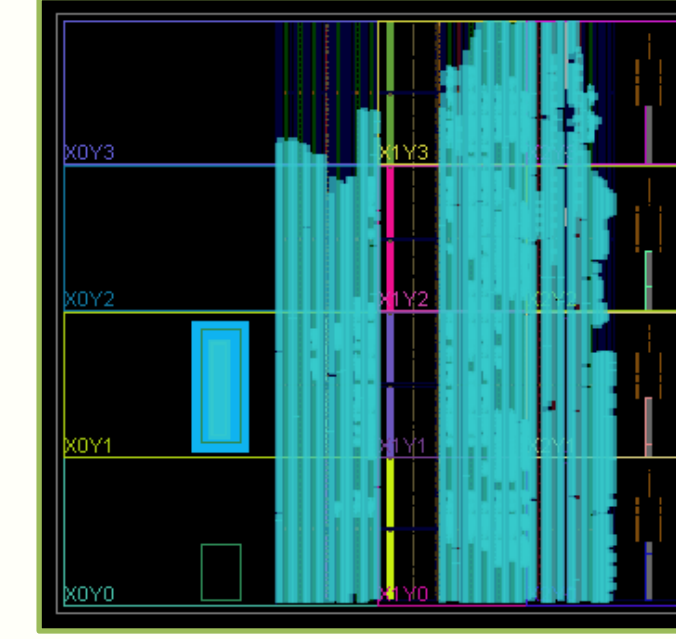


TABLE 5 Hardware Utilization and Timing

| DESIGN   | LUT     | LUT % | FF      | FF %  | BRAM | BRAM % | DSP | DSP % | WNS   | TIMING |
|----------|---------|-------|---------|-------|------|--------|-----|-------|-------|--------|
| SRCNN_42 | 57,820  | 49.37 | 69,404  | 29.63 | 73   | 50.69  | 145 | 11.62 | 0.589 | Met    |
| SRCNN_88 | 104,330 | 89.08 | 129,691 | 55.37 | 112  | 77.78  | 577 | 46.23 | 1.106 | Met    |
| SRCNN_84 | 40,980  | 34.99 | 79,463  | 33.92 | 33   | 22.92  | 397 | 31.81 | 2.550 | Met    |

URAM usage is 5 blocks (7.81%) for all three designs and is omitted here to keep the table compact for report capture.

# 4 Results & Conclusion

Demo: Test\_1.img Evaluation

✂ Verification (PSNR)

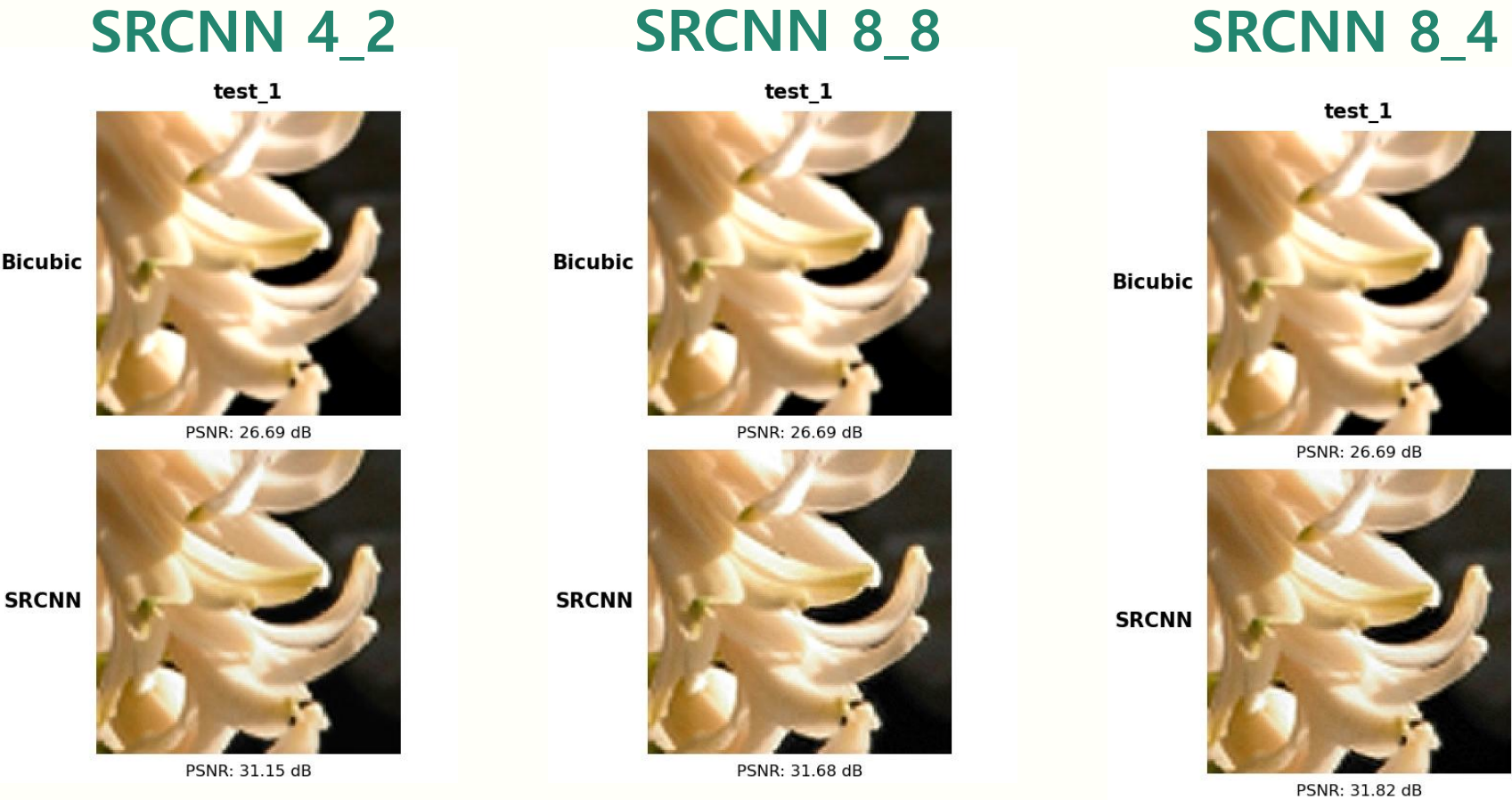


TABLE 6 Demo PSNR Results

| DESIGN   | TEST IMAGE | BICUBIC PSNR (DB) | SRCNN PSNR (DB) | IMPROVEMENT (DB) |
|----------|------------|-------------------|-----------------|------------------|
| SRCNN_42 | test_1     | 26.69             | 31.15           | +4.46            |
| SRCNN_88 | test_1     | 26.69             | 31.68           | +4.99            |
| SRCNN_84 | test_1     | 26.69             | 31.82           | +5.13            |



# 4 Results & Conclusion

## ◆ Built a **latency-oriented SRCNN accelerator**

→ Used channel-wise computation with architecture-specific latency reduction

## ◆ **Recursive Architecture**

→ SRCNN 4\_2: reduced latency with 2-pixel computation

→ SRCNN 8\_8: implemented 64-PE channel-wise parallelism without folding

## ◆ **Streamline Architecture**

→ SRCNN 8\_4: achieved the lowest latency through layer overlap

## ◆ **RTL Optimization**

→ Reduced LUT/BRAM usage and met timing through line-buffer and PE-pipeline tuning



# Thank you

